

Contents

区块链基础知识入门	2
第一章：什么是区块链	2
1.1 用银行账本来理解区块链	2
1.2 区块链的核心特点	2
第二章：什么是以太坊	2
2.1 以太坊 = 世界计算机	2
2.2 智能合约：自动执行的程序	3
第三章：数字货币基础	3
3.1 什么是代币 (Token)	3
3.2 常见代币类型	4
第四章：地址和钱包	4
4.1 什么是地址	4
4.2 为什么地址是 20 字节	5
4.3 钱包的作用	5
第五章：交易是如何工作的	5
5.1 发送交易的过程	5
5.2 Gas 费用 (手续费)	6
第六章：什么是 DeFi (去中心化金融)	7
6.1 传统金融 vs DeFi	7
6.2 DeFi 的核心应用	7
第七章：什么是 DEX 聚合器	8
7.1 为什么需要聚合器	8
7.2 聚合器如何工作	8
第八章：编程概念基础	9
8.1 什么是函数	9
8.2 什么是智能合约函数	9
8.3 函数调用编码	9
第九章：数据格式基础	10
9.1 十六进制数字	10
9.2 字节和地址	10
第十章：哈希函数基础	11
10.1 什么是哈希	11
10.2 哈希在区块链中的应用	11
第十一章：路径查找基础	11
11.1 什么是路径查找	11
11.2 为什么要多跳交易	12
第十二章：实际操作示例	12
12.1 完整的交易流程	12
12.2 智能合约如何工作	13
总结：现在你已经准备好了！	14
基础概念	14
技术细节	14
DeFi 概念	14

区块链基础知识入门

这是一份专门为新手准备的基础文档，看完后你就能理解我们之前讨论的所有复杂概念。

第一章：什么是区块链

1.1 用银行账本来理解区块链

想象一下传统银行：

传统银行账本（中心化）：

银行 ← 只有银行能修改账本

账本
张三：100
李四：50

区块链账本（去中心化）：

区块链账本：

节点A	节点B	节点C	
账本副本	账本副本	账本副本	← 每个人都有相同的账本
张三：100	张三：100	张三：100	
李四：50	李四：50	李四：50	

1.2 区块链的核心特点

1. **去中心化**：没有银行，大家共同维护账本
2. **透明**：所有交易记录公开可查
3. **不可篡改**：一旦记录，很难修改
4. **全球通用**：24 小时运行，无地域限制

第二章：什么是以太坊

2.1 以太坊 = 世界计算机

传统计算机：

你的电脑

程序 ← 只能在你的电脑上运行

以太坊：

节点A	节点B	节点C	
智能合约	智能合约	智能合约	← 程序在全球成千上万台电脑上运行
程序	程序	程序	

2.2 智能合约：自动执行的程序

想象一个自动售货机：

```
// 传统售货机（需要人工操作）
if（用户投入 1 元 && 按下可乐按钮）{
    人工检查钱是否足够；
    人工决定是否给可乐；
    人工找零；
}

// 智能合约（自动执行）
if（用户发送 1 ETH && 调用 buyToken 函数）{
    自动检查余额是否足够；           // 代码自动执行
    自动转账代币给用户；           // 无需人工干预
    自动扣除 ETH；                 // 透明公开
}
```

第三章：数字货币基础

3.1 什么是代币（Token）

代币就像游戏币，但在区块链上：

游戏厅：

游戏币：100个 ← 只能在这个游戏厅使用
只能玩游戏

区块链代币：

USDT: 1000个 ← 全球通用
可以：
- 转账给别人
- 换成其他币
- 买商品

3.2 常见代币类型

```
const 主要代币 = {  
  ETH: {  
    作用: " 以太坊网络的燃料",  
    比喻: " 汽车的汽油",  
    用途: " 支付交易手续费"  
  },  
  
  USDT: {  
    作用: " 稳定币, 价值锚定美元",  
    比喻: " 数字美元",  
    用途: " 保值、交易中介"  
  },  
  
  USDC: {  
    作用: " 另一种稳定币",  
    比喻: " 另一种数字美元",  
    用途: " 和 USDT 类似"  
  }  
};
```

第四章：地址和钱包

4.1 什么是地址

地址就像银行账户号码：

```
// 银行账户  
const 银行账户 = {  
  账户号: "6222021234567890123",  
  作用: " 接收和发送人民币",  
  长度: "19 位数字"  
};  
  
// 以太坊地址  
const 以太坊地址 = {  
  地址: "0xdac17f958d2ee523a2206206994597c13d831ec7",
```

```

    作用: " 接收和发送数字货币",
    长度: "42 个字符 (20 字节)",
    特点: " 以 0x 开头"
};

```

4.2 为什么地址是 20 字节

```

// 地址生成过程 (简化管理)
const 生成地址 = {
    第一步: " 创建公钥私钥对 (像生成密码) ",
    第二步: " 对公钥进行哈希运算",
    第三步: " 取哈希结果的最后 20 字节",
    第四步: " 加上 0x 前缀, 变成地址"
};

// 为什么是 20 字节?
const 原因 = {
    安全性: "20 字节 =  $2^{160}$  种可能 地球上所有沙粒的数量",
    效率: " 不会太长 (用户友好), 不会太短 (足够安全) ",
    兼容性: " 与以太坊存储系统完美配合"
};

```

4.3 钱包的作用

```

// 物理钱包
const 物理钱包 = {
    功能: " 装纸币和硬币",
    风险: " 丢了钱就没了"
};

// 数字钱包
const 数字钱包 = {
    功能: " 管理私钥 (密码) ",
    重要性: " 私钥 = 对资产的完全控制权",
    助记词: "12 个英文单词, 用来恢复钱包",
    风险: " 助记词泄露 = 资产全部丢失"
};

```

第五章：交易是如何工作的

5.1 发送交易的过程

```

// 张三给李四转账 100 USDT
const 转账过程 = {
    第一步: {

```

```

    操作: " 张三在钱包里输入李四的地址和金额",
    数据: {
      from: "0x 张三的地址...",
      to: "0x 李四的地址...",
      amount: "100 USDT"
    }
  },

  第二步: {
    操作: " 钱包用张三的私钥签名这笔交易",
    目的: " 证明这笔交易确实是张三发起的"
  },

  第三步: {
    操作: " 交易被广播到以太坊网络",
    过程: " 全世界的节点都会收到这个交易"
  },

  第四步: {
    操作: " 矿工验证交易并打包到区块",
    检查: [" 张三账户余额是否足够", " 签名是否正确", " 地址格式是否正确"]
  },

  第五步: {
    操作: " 交易确认, 李四收到 100 USDT",
    结果: " 区块链账本更新"
  }
};

```

5.2 Gas 费用（手续费）

```

// 为什么需要 Gas 费
const Gas 费解释 = {
  比喻: " 就像寄快递需要付邮费",
  作用: " 奖励矿工处理你的交易",
  计算: "Gas 用量 × Gas 价格 = 总手续费",

  实例: {
    简单转账: "21,000 Gas × 30 Gwei = 0.00063 ETH   $1.5",
    复杂操作: "200,000 Gas × 30 Gwei = 0.006 ETH   $15"
  }
};

// Gas 价格和网络拥堵
const 网络状况 = {

```

```

    空闲时间: "Gas 价格低 (10-20 Gwei) ",
    繁忙时间: "Gas 价格高 (100-500 Gwei) ",

    比喻: " 就像打车: ",
    平时: " 价格正常",
    高峰期: " 价格翻倍"
};

```

第六章：什么是 DeFi（去中心化金融）

6.1 传统金融 vs DeFi

```

// 传统金融
const 传统银行 = {
    存钱: " 需要去银行网点, 填表格, 等审批",
    借钱: " 需要抵押、担保、信用检查",
    换汇: " 只能在银行营业时间, 汇率由银行决定",
    限制: " 地域限制、时间限制、身份限制"
};

// DeFi
const DeFi 金融 = {
    存钱: " 连接钱包, 点击按钮, 立即生效",
    借钱: " 抵押数字资产, 自动借贷",
    换汇: "24 小时交易, 汇率由市场决定",
    优势: " 全球通用、无需身份、透明公开"
};

```

6.2 DeFi 的核心应用

```

const DeFi 应用类型 = {
    DEX: {
        全称: " 去中心化交易所",
        作用: " 用户直接交换代币",
        代表: "Uniswap、SushiSwap",
        比喻: " 自动外币兑换机"
    },

    借贷协议: {
        作用: " 存币赚利息, 抵押借币",
        代表: "Compound、Aave",
        比喻: " 自动运行的银行"
    },

    稳定币: {

```

```

        作用: " 价格稳定的数字货币",
        代表: "USDT、USDC、DAI",
        比喻: " 数字版美元"
    }
};

```

第七章：什么是 DEX 聚合器

7.1 为什么需要聚合器

// 问题: 同一个商品在不同商店价格不同

```

const 购物对比 = {
    商店 A: "iPhone 6000 元",
    商店 B: "iPhone 5800 元",
    商店 C: "iPhone 5900 元",

```

```

    聪明做法: " 比价后去最便宜的商店买"
};

```

// DEX 的情况也一样

```

const DEX 价格对比 = {
    Uniswap: "1 ETH = 2000 USDT",
    SushiSwap: "1 ETH = 2010 USDT", // 更好的价格
    Curve: "1 ETH = 1995 USDT",

```

```

    聚合器的作用: " 自动找到最好的价格"
};

```

7.2 聚合器如何工作

// 用户想用 10000 USDT 买 ETH

```

const 聚合器工作流程 = {
    第一步: {
        操作: " 扫描所有 DEX 的价格",
        结果: {
            Uniswap: " 可以买到 4.95 ETH",
            SushiSwap: " 可以买到 5.02 ETH",
            Curve: " 可以买到 4.90 ETH"
        }
    },

    第二步: {
        操作: " 计算最优分配方案",
        结果: "SushiSwap 买 5000USDT 的 ETH + Uniswap 买 5000USDT 的 ETH"
    },

```



```

    第三步: {
        操作: " 一键执行所有交易",
        用户体验: " 用户只需点击一次, 系统自动完成所有操作"
    }
};

```

第八章：编程概念基础

8.1 什么是函数

```

// 函数就像一个机器
function 榨汁机(水果类型, 数量) {
    if (水果类型 === " 苹果") {
        return 数量 + " 杯苹果汁";
    } else if (水果类型 === " 橙子") {
        return 数量 + " 杯橙汁";
    }
}

// 使用函数
const 结果 = 榨汁机(" 苹果", 2); // 返回 "2 杯苹果汁"

```

8.2 什么是智能合约函数

```

// 智能合约中的函数
contract 代币合约 {
    // 转账函数
    function transfer(address 收款人, uint256 金额) public {
        // 检查余额是否足够
        // 扣除发送方余额
        // 增加接收方余额
    }

    // 查询余额函数
    function balanceOf(address 用户) public view returns (uint256) {
        // 返回用户的代币余额
    }
}

```

8.3 函数调用编码

```

// 人类理解的函数调用
transfer("0x 李四的地址", 100);

```

```
// 计算机理解的函数调用 (16 进制编码)
"0xa9059cbb" + "000000000000000000000000 李四的地址" + "00000000000000000000000000000000"

// 解释:
// 0xa9059cbb = transfer 函数的标识符 (函数选择器)
// 后面的长串 = 编码后的参数 (地址和金额)
```

第九章：数据格式基础

9.1 十六进制数字

```
// 我们平时用十进制 (0-9)
const 十进制 = 255;

// 计算机常用十六进制 (0-9, A-F)
const 十六进制 = "0xFF"; // 和 255 相等

// 转换对照表
const 对照表 = {
  10: "A", 11: "B", 12: "C",
  13: "D", 14: "E", 15: "F"
};

// 为什么用十六进制?
// 1. 更紧凑: 255 用十六进制只需要 2 位 (FF)
// 2. 便于计算机处理: 1 字节 = 2 个十六进制字符
```

9.2 字节和地址

```
// 1 字节 = 8 位 = 2 个十六进制字符
const 字节示例 = {
  "1 字节": "FF", // 255
  "2 字节": "FFFF", // 65535
  "20 字节": "dac17f958d2ee523a2206206994597c13d831ec7" // 以太坊地址
};

// 以太坊地址解析
const 地址解析 = {
  完整地址: "0xdac17f958d2ee523a2206206994597c13d831ec7",
  分解: {
    前缀: "0x", // 表示这是十六进制
    实际地址: "dac17f958d2ee523a2206206994597c13d831ec7", // 40 个字符 = 20 字节
    字节数: 20
  }
};
```

第十章：哈希函数基础

10.1 什么是哈希

// 哈希就像指纹识别

```
const 哈希概念 = {  
  输入: " 任意长度的数据",  
  输出: " 固定长度的'指纹'",  
  特点: " 输入相同, 输出必定相同; 输入稍有不同, 输出完全不同"  
};
```

// 实际例子

```
const 哈希示例 = {  
  输入 1: "hello",  
  输出 1: "2cf24dba4f21d4288094e9b3528e5cd04fcfb84b8ff05a89e6e6c9ae5969",  
  
  输入 2: "Hello", // 只改了一个大小写  
  输出 2: "185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969" // 完全不同  
};
```

10.2 哈希在区块链中的应用

```
const 哈希应用 = {  
  地址生成: {  
    过程: " 公钥 → 哈希 → 地址",  
    目的: " 从长的公钥生成短的地址"  
  },  
  
  函数识别: {  
    过程: " 函数签名 → 哈希 → 函数选择器",  
    例子: "transfer(address,uint256) → 哈希 → 0xa9059cbb"  
  },  
  
  数据验证: {  
    过程: " 交易数据 → 哈希 → 交易 ID",  
    目的: " 确保数据没有被篡改"  
  }  
};
```

第十一章：路径查找基础

11.1 什么是路径查找

// 就像导航找路

```
const 导航类比 = {
```

```

    起点: " 家",
    终点: " 公司",
    可能路线: [
        " 家 → 地铁站 → 公司 (15 分钟, 5 元) ",
        " 家 → 公交站 → 换乘 → 公司 (25 分钟, 2 元) ",
        " 家 → 打车 → 公司 (10 分钟, 30 元) "
    ],
    最优选择: " 根据你的需求 (时间优先/费用优先) "
};

// DEX 交易路径
const 交易路径 = {
    起点: "USDT",
    终点: "ETH",
    可能路线: [
        "USDT → ETH (直接交易, 汇率: 1995) ",
        "USDT → USDC → ETH (两步交易, 汇率: 2010) ",
        "USDT → DAI → WETH → ETH (三步交易, 汇率: 2015) "
    ],
    最优选择: " 汇率最高的路径"
};

```

11.2 为什么要多跳交易

```

// 为什么不能直接交易?
const 流动性问题 = {
    问题: " 某些交易对没有足够的流动性",

    例子: {
        直接交易: "A 币 → B 币 (流动性不足, 汇率很差) ",
        多跳交易: "A 币 → USDT → B 币 (每一步流动性充足, 整体汇率更好) "
    },

    比喻: " 在小城市换外币 vs 在银行换外币",
    小城市: " 汇率差, 手续费高",
    银行: " 汇率好, 但需要多跑一趟"
};

```

第十二章：实际操作示例

12.1 完整的交易流程

```

// 用户想用 1000 USDT 买 ETH
const 完整流程 = {
    第一步: {

```

```

        用户操作: " 在 OpenOcean 输入: 1000 USDT → ETH",
        系统动作: " 扫描所有 DEX 价格"
    },

    第二步: {
        系统计算: " 找到最佳路径: 500 USDT 在 Uniswap + 500 USDT 在 SushiSwap",
        预期收益: "0.502 ETH (比单一 DEX 多 0.02 ETH) "
    },

    第三步: {
        用户确认: " 点击确认按钮",
        钱包弹窗: "MetaMask 弹出签名确认"
    },

    第四步: {
        合约执行: "OpenOcean 合约自动分配订单到两个 DEX",
        交易完成: " 用户收到 0.502 ETH"
    }
};

```

12.2 智能合约如何工作

```

// 简化的聚合器合约
contract 简化聚合器 {
    function 交换 (
        address 输入代币,
        address 输出代币,
        uint256 输入数量,
        路径 [] memory 交易路径
    ) public {
        // 第 1 步: 检查用户授权
        require(用户已授权代币给合约);

        // 第 2 步: 从用户账户转入代币
        从用户转入 (输入代币, 输入数量);

        // 第 3 步: 执行每个路径的交易
        for (每个路径 in 交易路径) {
            在指定 DEX 上交易 (路径.DEX 地址, 路径. 数量);
        }

        // 第 4 步: 将所有结果转给用户
        转给用户 (输出代币, 获得的总数量);
    }
}

```

总结：现在你已经准备好了！

看完这份基础文档，你现在应该能理解：

基础概念

- 区块链是什么
- 以太坊是什么
- 智能合约如何工作
- 代币和地址的概念

技术细节

- 为什么地址是 20 字节
- 函数选择器如何计算
- 哈希函数的作用
- 十六进制数据格式

DeFi 概念

- 什么是 DEX
- 为什么需要聚合器
- 路径查找的意义
- 流动性和价格的关系

实际应用

- 交易是如何执行的
- 智能合约如何自动化
- 为什么需要 Gas 费
- 多 DEX 分单的优势

现在回去看我们之前写的所有文档，你应该能够：1. 理解每个技术术语的含义 2. 知道复杂数据编码的原理 3. 明白 DEX 聚合器的工作流程 4. 看懂智能合约的设计逻辑

如果还有任何不理解的地方，随时问我！